

PipelineInsights — shared context. This is background, not a task. Do not write code in response to this message. Acknowledge briefly and wait for the task card.

What the project is

A multi-sport player-prop prediction pipeline. Tennis is the first sport. MLB is next as a team-sport bridge, then the architecture gets generalized, then NFL.

Architecture rules, non-negotiable

Models never read raw data. They read only from the single feature table `player_game_features`. This is what makes the same model code portable to another sport.

Sport-specific work touches only: data sources, ETL scripts, raw tables, and feature engineering. Everything downstream — models, ensemble, the predictions table, backtesting, the dashboard — stays unchanged when a sport is added.

The dashboard reads only from the view `vw_fact_player_prop_odds`. It never queries base tables directly.

Backtesting always uses time-based train/test splits: train on earlier dates, test on later dates. Never random splits. Random splits let the model see the future and produce results that look excellent and mean nothing.

Never overwrite an existing `modelversion`. Bump it on any change to features, model type, hyperparameters, or ensemble weights. Old versions stay for side-by-side comparison.

Current state of the code, verified by reading it

`models/baseline.py` line 297 defines `upsert_predictions(connection, predictions)`. It inserts into `player_prop_predictions` with `ON CONFLICT (player_id, match_id, prop_type, modelversion)`.

`models/gbm.py` imports that function at line 20 and calls it at line 456, gated on `if not dry_run`. `models/ensemble.py` imports it at line 21 and calls it at line 432.

So all three models already persist predictions through one shared writer. This does not need to be built.

`player_prop_predictions` schema, from `db/migrations/003_predictions.sql`:

- `UNIQUE (player_id, match_id, prop_type, modelversion)`
- Foreign key (`match_id, player_id`) into `matches`, so a prediction cannot exist without its match row
- Migration 013 adds a `CHECK` named `prediction_interval_ordered` requiring `lowerci >= 0` and `lowerci <= prediction <= upperci`

`vw_fact_player_prop_odds`, currently defined in `db/migrations/008_checkpoint4_resolution.sql`, joins predictions to players and matches with inner joins, and to odds with a `LEFT JOIN`. This matters: predictions appear in the view even when no odds exist, with `line`, `edge`, `side`, and `normalized_edge` all `NULL`. An empty dashboard is therefore never caused by missing odds.

`scripts/seed_demo.py` seeds eight fake players with `demo_id` prefixes — Ava Chen, Mateo Silva, Jordan Brooks, Noah Okafor, Sofia Rossi, Mina Park, Elena Petrova, Layla Morgan. It also seeds fake rows into odds and `player_prop_predictions`. Its `DELETE` statements are correctly scoped to `player_id LIKE 'demo_%%'`, so running it cannot erase real data.

`evaluation/checkpoint12.py` computes backtest metrics and writes a report and a model card to files. It does not persist per-row predictions. Its market-comparison query correctly excludes demo data with `WHERE prediction.player_id NOT LIKE 'demo_%%' and market.provider <> 'demo'`.

The odds situation

Prop odds capture began 2026-09-17. There is no earlier prop odds history and it cannot be purchased, so backtesting predictions against historical market lines is impossible. Do not attempt it and do not simulate it.

The captured archive so far is entirely WTA. The reason is verified: the ATP tour has not played since the US Open final on 2026-09-13, and the capture started after that. ATP main-tour markets have never been observed under this capture, so no conclusion should be drawn about which ATP prop markets the books offer.

The feature pipeline filters `WHERE player.tour = 'ATP'`. The model's targets are aces, `double_faults`, and `service_games`.

Model results as last measured

A real backtest over 2015–2026, roughly 66,610 rows per target across 129 monthly folds, reported the GBM beating the rolling-average baseline by roughly 20–28% on mean absolute error depending on the target. The ensemble currently loses to the GBM on all three targets.

Treat those numbers as the last reported run, not as gospel. If your work changes them, report the change.

How to report

The person you are working for is a coding and ML novice who reads the actual code and checks claims against it. Explain the reasoning behind technical choices in plain language. Define jargon in a few words when you first use it.

Never report an item as complete when it is not. If you skipped something, say so and say why. A report listing four completions and two honest omissions is more useful than one claiming six.

